

proxy_concurrency_test.sh — operator guide

tests/concurrency/ proxy_concurrency_test.sh — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Authored + parse-clean (sh -n + bash -n). Runs live against the running HTTP forward proxy (localhost:53128) AND the SOCKS5 proxy (localhost:51080); honest SKIP (§11.4.3) when no proxy is listening or the echo oracle is unreachable directly.

Companion (§11.4.18) to the in-source documentation block at the top of the script.

Overview

§11.4.169 **concurrency / atomicity** suite for the LIVE proxy data plane. It proves the proxy is SAFE under genuinely-**simultaneous** callers — the §1 "always consider concurrent callers" concern — with **no cross-talk**: no response served to the wrong client, no truncation, no corruption.

The suite launches a HIGH number of clients (default **40**) that all park on a start **barrier** and are released **at once**, MIXING the HTTP forward proxy (http://localhost:53128) AND the SOCKS5 proxy (socks5h://localhost:51080) round-robin. Every client fetches a **distinct identifiable resource**: a per-client unique token — HPCXT_<run-nonce>_C<i>_END — echoed back in the response body by the echo endpoint. Cross-talk is then **mechanically detectable** — each client MUST receive ITS OWN token and NO OTHER client's token. The trailing _END sentinel doubles as a truncation detector.

It PASSes only when **every** simultaneous client received its own **correct, complete, distinct** response, citing captured per-client evidence (§11.4.69) — a metadata-only PASS is refused. A foreign token (response served to the wrong client) or a corrupt/truncated

body is a **hard FAIL** (§11.4.68). This is **DISTINCT** from tests/stress/proxy_forward_stress.sh, which drives sequential load plus a same-URL HTTPS-CONNECT burst on ONE transport and checks only HTTP codes — it does not test simultaneity across transports nor cross-talk.

Prerequisites

- Committed library tests/lib/evidence.sh (sourced — provides `_code_in`, `port_is_listening`, `ab_pass_with_evidence`, `ab_skip_with_reason`).
- curl (with SOCKS5 support), awk, sed, sort, grep, tr, POSIX sh, date.
- A running HTTP forward proxy on `HTTP_PROXY_PORT` and/or a SOCKS5 proxy on `SOCKS_PROXY_PORT` for a non-SKIP run (both listening ⇒ a genuinely mixed run).
- A reachable **echo endpoint** that reflects the `__TOKEN__` substring back in its response body (default `postman-echo.com`; a self-hosted echo is the ideal target under heavy concurrent load — see Edge cases).
- Write access to `qa-results/` (gitignored).

Usage examples

- Default run against `localhost:53128` (HTTP) + `localhost:51080` (SOCKS5): `bash tests/concurrency/proxy_concurrency_test.sh`
- Conductor invocation under host-safety caps (§12.6):
`GOMAXPROCS=2 nice -n 19 ionice -c 3 bash tests/concurrency/proxy_concurrency_test.sh`
- Higher simultaneity against a self-hosted echo (clean PASS/FAIL under load): `CONC_CLIENTS=60`
`CONC_ECHO_URL_TEMPLATE=http://127.0.0.1:8080/echo?t=__TOKEN__`
`bash tests/concurrency/proxy_concurrency_test.sh`

Env knobs: `HTTP_PROXY_URL` (default `http://localhost:53128`), `HTTP_PROXY_PORT` (default 53128), `SOCKS_PROXY_URL` (default `socks5h://localhost:51080`), `SOCKS_PROXY_PORT` (default 51080), `CONC_CLIENTS` (default 40, clamped 2..80), `CONC_ECHO_URL_TEMPLATE` (default `https://postman-echo.com/get?htok=__TOKEN__` — **MUST** contain the `__TOKEN__` placeholder and reflect it in the body), `CONC_EXPECT` (default 200), `CONC_ENDPOINT_LIMIT_CODES` (default 429 500 502 503 504), `CURL_MAX_TIME` (default 20), `CONC_BARRIER_ARM_SECS` (default 1), `CONC_EVIDENCE_DIR` (default `qa-results/concurrency/proxy_concurrency_<ts>`).

Edge cases

- **Every simultaneous client got its own correct distinct token** → PASS, citing `clients.tsv`. Exit 0.
- **Any client received a FOREIGN client's token** (cross-talk / a response served to the wrong client / body corruption) → hard FAIL (§11.4.68), always, regardless of endpoint health. Exit 1.
- **A concurrent proxied request was dropped or garbled** (000, or a 200 with the own token missing / truncated) **while the echo is reachable directly** → FAIL — a real proxy defect under concurrency (§11.4.68 — a listening-but- dropping proxy never fail-opens to SKIP). Exit 1.
- **The echo endpoint is unreachable DIRECTLY** (baseline oracle cannot be established) → SKIP
(`network_unreachable_external`) — the cross-talk test cannot run; a third-party outage is not a proxy defect (§11.4.1). Exit 3.
- **The echo endpoint rate-limited / saturated under our own concurrent load** (only 429/5xx endpoint-limit shortfalls, no cross-talk, no proxy drop) → SKIP
(`network_unreachable_external`) — a third-party limit, not a proxy defect. Point `CONC_ECHO_URL_TEMPLATE` at a self-hosted echo for a clean PASS/FAIL under high concurrency. Exit 3.
- **No proxy listening on either port** → SKIP
(`topology_unsupported`). Exit 3.
- **Deadlock guard**: no fetch can exceed `CURL_MAX_TIME` (reports 000); the barrier wait is itself bounded (~30s ceiling) so a worker never blocks forever.

Internal behaviour

- `#!/usr/bin/env bash`, POSIX-clean body (`sh -n + bash -n`, §11.4.67); no bash arrays / `[[ ]]` / `<<<` / process-substitution.
- Best-effort self-caps: exports `GOMAXPROCS=2` and applies `renice 19 + ionice -c 3` to its own PID (the conductor SHOULD ALSO wrap with the caps).
- Preflight: probes `port_is_listening` for each proxy and runs a DIRECT baseline echo probe with sentinel token `c0` to establish the cross-talk oracle before any load is launched. Honest SKIP gates fire here.
- Each worker parks on a `G0` barrier file, then performs ONE distinct fetch and writes its `%{http_code}`, proxy label, token, and body to its OWN files — a verdict never rests on a background job's exit status (the B3 anti-bluff pattern). Workers are released simultaneously by touching `G0`.
- Classification (main thread): each client's body is scanned for every `HPCXT_<nonce>_C<num>_END` token. Correct = the only number present is that client's index AND the HTTP code is expected; a foreign number → CROSSTALK; an endpoint-limit code

- `ENDPOINT_LIMIT`; otherwise → `PROXY_DROP`. The per-client verdict lands in `clients.tsv`; the summary in `concurrency.evidence`.
- `trap ... EXIT INT TERM` signals ONLY this script's worker PIDs (never kill 0) and removes the scratch dir (§11.4.14); evidence files are preserved.
 - Resources: shell + curl only, simultaneous clients clamped ≤ 80, well under the §12.6 60% host-memory ceiling. It NEVER touches operator resources (`wg0-mullvad`, `lava-*`, `whoami:58080`) and never stops/starts any container.

Related

- `tests/lib/evidence.sh` — sourced anti-bluff evidence library.
- `tests/stress/proxy_forward_stress.sh` — the sibling **stress** suite (sequential + same-URL HTTPS-CONNECT burst on the HTTP transport); this concurrency suite is deliberately DISTINCT (simultaneity + mixed transports + cross-talk).
- `challenges/scripts/proxy_forward_http_challenge.sh`, `challenges/scripts/proxy_socks5_challenge.sh` — the functional forward / SOCKS5 Challenges this concurrency suite complements.
- Constitution §11.4.169 / §1 (concurrent callers) / §11.4.69 / §11.4.68 / §11.4.1 / §11.4.50 / §11.4.107 / §12.6.

Last verified

2026-07-01 — authored; `sh -n + bash -n parse-clean`; cross-talk oracle extraction validated in isolation (own-token, foreign-token, truncation, empty, and the C1-vs-C10 index-boundary cases). Live simultaneous mixed-transport load is exercised by the conductor against the running proxies (§11.4.119).